



FET1052-C、OK1052-C

Embedded Development Platform

Linux Manual

Rev.1.0

2018/08/06

注意事项与维护



1、注意事项

请勿带电插拔核心板及外围模块！

请遵循所有标注在产品上的警示和指引信息。

请保持本产品干燥。如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干。

- 使用中注意本产品的通风散热，避免温度过高造成元器件损坏。
- 请勿在多尘、脏乱的环境中使用或存放本产品。
- 请勿将本产品应用在冷热交替环境中，避免结露损坏元器件。
- 请勿粗暴对待本产品，跌落、敲打或剧烈晃动都可能损坏线路及元器件。
- 请勿使用有机溶剂或腐蚀性液体清洗本产品。
- 请勿自行修理、拆卸本公司产品，如产品出现故障请及时联系本公司进行维修。
- 擅自修改或使用未经授权的配件可能损坏本产品，由此造成的损坏将不予以保修。

2、售后维修

如产品使用过程中出现硬件故障可根据售后服务政策进行维修；

服务政策：参见官方网站 www.forlinx.com 售后服务说明；

地址：河北省保定市高开区向阳北大街 2699 号飞凌嵌入式四楼西厅

联系人：售后维修部

电话：0312-3102637 邮编：071000

邮寄须知：建议使用顺丰、圆通或韵达，且不接收任何到付。

技术支持与定制

1、技术支持范围

- 1.1 本公司产品的软、硬件资源提供情况咨询；
- 1.2 本公司产品的软、硬件手册使用过程中遇到的问题；
- 1.3 本公司提供的 OEM、ODM 售后技术支持；
- 1.4 本公司产品的故障判断及售后维修服务。

2、技术讨论范围

- 2.1 源码的修改以及理解；
- 2.2 操作系统如何移植；
- 2.3 用户在自行修改以及开发中遇到的软硬件问题；

注：以上三点虽不属于技术支持范围，但我公司会尽力为用户提供帮助，如仍然没能解决您的问题，敬请谅解。

3、技术支持方式

3.1 电话: 0312-3119192 / 0312-3102619

3.2 论坛: bbs.witech.com.cn

3.3 邮箱:

Linux 技术支持: linux@forlinx.com

Android 技术支持: android@forlinx.com

硬件技术支持: hardware@forlinx.com

3.4 知识库: bbs.witech.com.cn/kb

4、技术支持时间

周一至周五: 上午 9:00—11:30, 下午 13:30—17:00;

公司按照国家法定节假日安排休息, 在此期间无法提供技术支持, 请将问题发送至邮箱或论坛技术支持区, 我们会在工作日尽快给您回复。

5、定制开发服务

我公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务, 以缩短您的产品开发周期。

了解定制流程: <http://www.forlinx.com/OEM.htm>

填写需求文档: <http://www.forlinx.com/docs/PR.docx>

发至项目邮箱: project@forlinx.com

资料更新与获取

1、资料的更新

产品相关资料会不断的完善更新，本手册内容亦然如此；当您在使用这些内容时，请确保其为最新状态。

2、更新后如何通知

飞凌嵌入式产品资料更新通知采用微信公众号推送，敬请关注！



订阅号

3、资料如何获取

3.1 网络下载：

请注册并登陆“bbs.witech.com.cn”找到“[开发板资料下载](#)”选择对应平台下载；
下载前请阅读《资料下载说明》：<http://bbs.witech.com.cn/thread-67932-1-1.html>。

3.2 光盘：

请联系我公司销售人员购买。

版权声明

本手册版权归保定飞凌嵌入式技术有限公司所有。未经本公司的书面许可，任何单位和个人无权以任何形式复制、传播、转载本手册的任何部分，违者将被追究法律责任。

更新记录

日期	手册版本	底板版本	核心板版本	更新内容
2018.06.07	V1.0	V1.2	V1.1	OK1052-C Linux 用户手册第一版。

目 录

注意事项与维护	- 2 -
1、注意事项	- 2 -
2、售后维修	- 2 -
技术支持与定制	- 2 -
1、技术支持范围	- 2 -
2、技术讨论范围	- 2 -
3、技术支持方式	- 3 -
4、技术支持时间	- 3 -
5、定制开发服务	- 3 -
资料更新与获取	- 4 -
1、资料的更新	- 4 -
2、更新后如何通知	- 4 -
3、资料如何获取	- 4 -
版权声明	- 4 -
更新记录	- 5 -
目 录	- 6 -
第一章 OK1052-C 软硬件资源介绍	- 7 -
1.1 Linux 4.5 系统软件资源介绍	- 7 -
1.2 SPI-Norflash 存储器分区表	- 7 -
第二章 使用前的准备	- 8 -
2.1 设置启动模式	- 8 -
2.2 设置超级终端	- 8 -
第三章 OK1052-C 平台 Linux 使用篇	- 10 -
3.1 SDHC TF 卡驱动测试	- 10 -
3.2 USB HOST 接口存储测试	- 11 -
3.3 USB OTG 接口存储测试	- 11 -
3.4 I2C 测试	- 11 -
3.5 以太网驱动测试	- 12 -
3.6 串口测试	- 13 -
3.7 LED 测试	- 14 -
3.8 按键测试	- 14 -
3.9 LCD 测试	- 15 -
3.10 电阻触摸测试	- 15 -
3.11 RTC 时钟测试	- 16 -
第四章 OK1052-C 平台 Linux 开发篇	- 17 -
4.1 版本说明	- 17 -
4.2 编译环境说明	- 17 -
4.3 编译环境搭建	- 17 -
4.4 编译应用	- 18 -
4.5 文件系统镜像制作	- 20 -
附录	- 21 -
F.1 使用 TF 卡启动/烧写镜像	- 21 -

第一章 OK1052-C 软硬件资源介绍

1.1 Linux 4.5 系统软件资源介绍

设备	设备名
micro SD/TF 卡驱动	/dev /mmcblk0pX
USB HOST 接口 U 盘	/dev/sda
USB OTG 接口 U 盘	/dev/sda
I2C EEPROM	/dev/i2c-0
以太网	eth0
串口	调试终端 /dev/ttyLP0 用户可用/dev/ttyLP1
LED GPIO OUT	/sys/devices/platform/leds/leds/led1/brightness
用户按键	/dev/input/event1
LCD FrameBuffer	/dev/fb0
Tsc 电阻触摸	/dev/input/event0
RTC	/dev/rtc0

1.2 SPI-Norflash 存储器分区表

下面表格是Linux操作系统的SPI-NorFlash存储器分区信息：

名称	偏移	大小	文件系统	内容
Bootloader (启动引导分区)	0x0	0x50000(320kB)	N/A	uboot
Bootloader (启动引导分区)	0x50000	0x20000(128kB)	N/A	uboot env
Kernel (启动分区)	0x70000	0x10000(64KB)	N/A	dtb
Kernel (启动分区)	0x80000	0x400000(4MB)	N/A	kernel
System 系统分区)	0x480000	0xB80000(11.5MB)	jffs2	系统文件

第二章 使用前的准备

2.1 设置启动模式

启动介质	拨码位置1	拨码位置2	拨码位置3	拨码位置4
SPI-NorFlash启动	1 OFF	2 OFF	3 OFF	4 OFF
TF启动/烧写系统	1 OFF	2 OFF	3 ON	4 OFF

说明：1，2，3，4 代表拨[码开关K2] 4个位置的4个拨码开关,断电状态拨动拨码开关。

2.2 设置超级终端

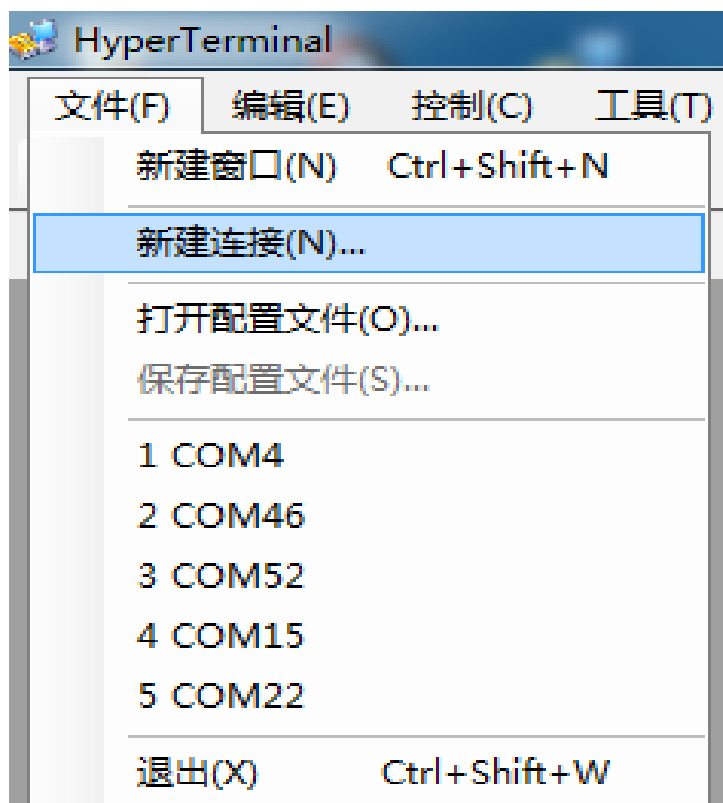
超级终端软件在 win7 系统需要从如下用户资料位置复制到 Win7 系统中。

📁 OK1052-C 用户资料\工具\Hyper Terminal.rar

问：如何查看内核的打印信息？

答：

1. PC 和开发板的 DEBUG 口通过双孔交叉 RS232 串口线连接。
2. 复制 [Hyper Terminal.rar](#) 到 windows 系统 d, 释放压缩包，找到 HyperTerminal.exe。（HyperTerminal 供学习使用）
3. 打开超级终端，文件->新建连接



根据您的串口连接进行选择。



设置完成后，启动就可以看到调试信息了。

第三章 OK1052-C 平台 Linux 使用篇

本节测试所用到的测试程序在飞凌提供的 Demo 存储在 TF 卡中，故不做文件来源说明，直接进行命令操作。

本节用到的测试程序源码路径：

 OK1052-C 用户资料\Linux\测试程序\testapp.zip

其中 testapp.zip 解压后 app-cmd 文件夹下有飞凌提供的命令行 Demo 的源码。

本节用到的测试程序可执行文件路径：

 OK1052-C 用户资料\Linux\测试程序\linoxrt_app

注意：

执行命令的过程中，根据不同情况，按“Ctrl+C”或“ESC”取消。

所有命令都是使用英文的字符输入。

3.1 SDHC TF 卡驱动测试

开发板支持 TF 卡热插拔。同时终端会打印关于卡的信息，由于存在很多种卡，显示的信息可能会有差别：

测试 SDHC TF 卡的方法：

将 4G SDHC TF 卡插入 TF 卡座

串口信息：

```
/ # mmc0: host does not support reading read-only switch, assuming write-enable
```

```
mmc0: new high speed SDHC card at address aaaa
```

```
mmcblk0: mmc0:aaaa SU04G 3.69 GiB
```

```
mmcblk0: p1
```

OK1052-C 支持手动 TF 卡挂载，默认挂载路径为：

```
/mnt/sdcard
```

挂载命令：

```
/ # mount -t vfat /dev/mmcblk0p1 /mnt/sdcard/
```

```
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be corrupt. Please run
```

```
fsck.
```

```
/ #
```

查看 TF 卡中的文件

```
/ # ls /mnt/sdcard/
```

```
helloworld          splash-rt1050-series_24.bmp
```

```
mxrt105x-evk.ini     uboot.env
```

```
rootfs.ulimage
```

往 TF 卡中写入文件，命令如下，写“helloworld”到 test.txt 文件中：

```
/ # echo "helloworld" > /mnt/sdcard/test.txt
```

```
/ # sync
```

读取 TF 卡中 test.txt 文件，命令如下：

```
/ # cat /mnt/sdcard/test.txt
```

```
helloworld
```

会读到刚才我们写入的 helloworld。

卸载 sd 卡

```
/ # umount /mnt/sdcard
```

此时拔出再插入 TF 卡到 TF 卡座，重新挂载，也会看到挂载成功，且能读取里面的内容。

3.2 USB HOST 接口存储测试

你可以在板载 USB HOST 接口上连接您的 USB 鼠标、USB 键盘、U 盘，并支持以上设备的热插拔。这里用挂载 U 盘为例进行演示，目前 U 盘测试支持到 32G，32G 以上并未测试。

终端会打印关于 U 盘的信息，由于存在很多种 U 盘，显示的信息可能会有差别：

步骤 1： 开发板启动后，连接 USB 接口 u 盘到飞凌开发板的 USB host 接口。

串口信息：

```
scsi 0:0:0:0: Direct-Access    TOSHIBA  TransMemory    PQ: 0 ANSI: 6
sd 0:0:0:0: [sda] 60620800 512-byte logical blocks: (31.0 GB/28.9 GiB)
sd 0:0:0:0: [sda] Write Protect is off
sd 0:0:0:0: [sda] Mode Sense: 4f 00 00 00
sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
sda: sda1
sd 0:0:0:0: [sda] Attached SCSI removable disk
```

步骤 2： 挂载目录

```
#mkdir /mnt/usbflash
/ # mount -t vfat /dev/sda1 mnt/usbflash/
```

串口信息：

```
FAT-fs (sda1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck
```

步骤 3： 查看 U 盘内容

```
/ # ls /mnt/usbflash/
```

串口信息输出 u 盘中的存储文件列表

卸载 u disk

```
/ # umount /mnt/usbflash
```

3.3 USB OTG 接口存储测试

使用 otg 转 host 线连接到 otg 口，插入 u 盘能读取 u 盘内容

步骤 1： 挂载目录

```
#mkdir /mnt/usbflash
/ # mount -t vfat /dev/sda1 mnt/usbflash/
```

串口信息：

```
FAT-fs (sda1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck
```

步骤 2： 查看 U 盘内容

```
/ # ls /mnt/usbflash/
```

串口信息 u 盘中的存储文件列表

卸载 u disk

```
/ # umount /mnt/usbflash
```

3.4 I2C 测试

I2C 总线测试使用如下，以 i2c 接口 EEPROM 存储器为例。

挂载命令：

```
/ # mount -t vfat /dev/mmcbkOp1 /mnt/sdcard/
```

步骤 1： 查看 i2c 总线编号

```
/ # /mnt/sdcard/imxrt_app/i2cdetect -l /dev/i2c-0
```

```
i2c-0    i2c          403f0000.lpc12c          I2C adapter
```

显示 总线编号 0

步骤 2: 查看 i2c 0 总线挂载的设备

```
/ # /mnt/sdcard/imxrt_app/i2cdetect -y 0
```

```
/mnt/sdcard # ./i2cdetect -y 0
```

```
0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
```

```
00:  -----
```

```
10:  ----- 1a -----
```

```
20:  -----
```

```
30:  --- 32 -----
```

```
40:  ----- UU ----- 4c -----
```

```
50: 50 -----
```

```
60:  -----
```

```
70:  -----
```

0x50 挂载有设备

步骤 3: 查看 0x50 地址设备内的数据

```
/ # /mnt/sdcard/imxrt_app/i2cdump -f -y 0 0x50
```

No size specified (using byte-data access)

```
0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
```

```
00: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
10: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
20: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
30: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
40: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
50: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
60: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
70: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
80: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
90: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
a0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
b0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
c0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
d0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
e0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

```
f0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

步骤 4: 设置挂载在 0 号 i2c 总线从设备地址是 0x50 的设备内部 0x00 地址上数据 0x01

```
/ # /mnt/sdcard/imxrt_app/i2cset -y 0 0x50 0x00 0x01
```

步骤 5: 读取挂载在 0 号 i2c 总线从设备地址是 0x50 的设备内部 0x00 地址上数据是 0x01

```
/ # /mnt/sdcard/imxrt_app/i2cget -y 0 0x50 0x00
```

```
0x01
```

3.5 以太网驱动测试

网络相关配置声明:

每个开发板的网络使用环境未必相同，本节测试示例中，网络环境如下。实际使用中，请按照实际网络环境自行进行配置，但需要 PC 与开发板在同一网段使用。

网络连接方式：开发板直接连接能够上网的路由器。

开发板 IP：192.168.1.163

路由器 IP：192.168.1.1

子网掩码：255.255.255.0

步骤 1：设置开发板的 IP 地址，您有以下两种设置方法可以选择。

方法一：命令行设置固定 IP 地址

```
# ifconfig eth0 up
```

```
# ifconfig eth0 192.168.1.163
```

（将 eth0 设置 IP 为 192.168.1.163）

```
# ifconfig
```

（查看当前网络状况）

命令行设置路由默认网关 IP 地址

```
# route add default gw 192.168.1.1
```

在/etc/resolv.conf 文件中配置 dns 服务信息(请以实际 dns 为准)

```
nameserver 222.222.222.222
```

```
nameserver 222.222.202.222
```

#打开网页测试

```
#rm index.html
```

```
# wget baidu.com
```

```
#ls index.html
```

```
index.html
```

说明：wget 命令用来从指定的 URL 下载文件。wget 非常稳定，它在带宽很窄的情况下和不稳定网络中有很强的适应性，如果是由于网络的原因下载失败，wget 会不断的尝试，直到整个文件下载完毕。如果是服务器打断下载过程，它会再次联到服务器上从停止的地方继续下载。这对从那些限定了链接时间的服务器上下载大文件非常有用。

方法二：命令行设置自动获取 IP 地址

如果您的开发板与路由器连接，且路由器支持 DHCP 自动 IP 地址分配可以在命令行输入：

```
# udhcpc -i eth0
```

用来动态获取 IP 地址，“-i” 参数用来指定网卡名称，飞凌开发板有线网络的网卡名称为 eth0。

/etc/resolv.conf 文件中有 dns 服务器信息会被自动添加。

步骤 2：网络连通测试

使用 ping 命令测试网络是否连通，以 ping 默认网关为例，请根据实际网络环境测试，命令如下：

```
# ping -I eth0 192.168.1.1
```

```
# ping -I eth0 baidu.com
```

#打开网页测试

```
#rm index.html
```

```
# wget baidu.com
```

```
#ls index.html
```

```
index.html
```

按键盘的 Ctrl + C 退出。

3.6 串口测试

1. OK1052-C 开发板中有 2 个串口，在开发板上的标示分别为 UAR1 UART2，串口 1 为调试串口，不需要测试。参考开发板原理图对串口的收发管脚进行短接，本文以 UART2 为例进行测试，其它串口测试方法相同。

(OK1052-C 设备是/dev/ttyLP1)。

1) UART1 口，232 电平，用做调试串口，在 Linux 中的设备名称： /dev/ttyLP0;

2) UART2 口，TTL 电平，用做普通串口，在 Linux 中的设备名称： /dev/ttyLP1;

2. 终端中输入如下命令

```
/ # mount -t vfat /dev/mmcbk0p1 /mnt/sdcard/  
/ # /mnt/sdcard/imxrt_app/uart /dev/ttyLP1
```

串口信息：

```
Welcome to TTYtest! Press Ctrl + 'c' to stop.
```

```
/dev/ttyLP1,creat thread 1026 sucess
```

```
/dev/ttyLP1,creat thread 2051 sucess
```

```
forlinx uart test.....bzforlinx uart test.....bzforlinx uart test.....bzno data within
```

1s.

```
forlinx uart test.....bzno data within 1s.
```

```
forlinx uart test.....bzno data within 1s.
```

```
^Cforlinx uart test.....bzSignHandler exit
```

```
send pthread exit
```

```
SignHandler exit
```

```
read pthread exit
```

```
SignHandler exit
```

```
/dev/ttyLP1,Send: 168 ,Receive: 168
```

```
main exit
```

测试程序发送的数据能够正常接收，表明测试成功！

3.7 LED 测试

OK1052-C 底板有用户 led 灯，可以控制灯引脚的高电平和低电平输出。

控制 led 管脚输出点亮

```
echo 1 > /sys/devices/platform/leds/leds/led1/brightness
```

控制 led 管脚输出熄灭

```
echo 0 > /sys/devices/platform/leds/leds/led1/brightness
```

3.8 按键测试

OK1052-C 开发板上有 3 个用户按键，K3 WakeUP/用户按键，键码为 A，K1 为 POR 电源复位按键，K4 为 ON/OFF 键，开关机作用。

3.8.1 K3 用户按键

本程序测试按键 K3

```
/ # hexdump /dev/input/event1
```

按下 OK1052-C 底板上的 k3 按键

串口打印类似信息：

```
0000030 01bd 0000 eccf 0007 0000 0000 0000 0000
```

3.8.2 K1 POR 按键

按下 POR 按键，开发板重启。

3.8.3 K4 ON/OFF 按键

长按超过 6s 按键，核心板关机，核心板灯 led 灭；此状态下长按按键超过 2s，核心板灯 led 亮开机。

注意：如果核心板处于关机状态，关掉底板电源开关，再上电，核心板仍然处于关机状态。关机状态能断电保存当前状态。

3.9 LCD 测试

OK1052-C 有 4.3 吋的 LCD 屏幕，可以在屏幕上边显示图片

```
/ # mount -t vfat /dev/mmcblk0p1 /mnt/sdcard/  
/ # cd /mnt/sdcard/imxrt_app/  
/ # ./fbwrite
```

LCD 显示如下：



3.10 电阻触摸测试

OK1052-C 有 4.3 吋的电阻触摸，测试如下：

```
/ # hexdump /dev/input/event0
```

串口打印类似信息：

```
0000040 000d 0000 9d12 0003 0000 0000 0000 0000
```

3.11 RTC 时钟测试

RTC 测试，主要通过使用date 和hwclock 工具设置软件时间、硬件rtc时间，测试当操作系统重启的时候，软件时钟读取RTC 时钟同步RTC时间到linux系统（**注意：确保板子上已经安装了纽扣电池**）。

设置时间命令帮助如下：

```
/ # date --help
```

串口信息：

```
BusyBox v1.24.2 (2018-06-28 13:46:43 CST) multi-call binary.
```

```
Usage: date [OPTIONS] [+FMT] [TIME]
```

设置时间如下命令：

```
/ # date -s 2018.06.28-14:16:00
```

串口信息：

```
Thu Jun 28 14:16:00 UTC 2018
```

说明：date 设置的格式：MMDDhhmm[[YY]YY][.ss]，MM 是月份，DD 是日期，hh 是小时，mm 是分钟，年份可以写 2 位的 YY 或 4 位 YYYY，秒可以用.ss 表示。

hwclock 查看下硬件 rtc 时间

```
/ # hwclock --help
```

串口信息：

```
BusyBox v1.24.2 (2018-06-28 13:46:43 CST) multi-call binary.
```

```
Usage: hwclock [-r] [-s] [-w] [-t] [-l] [-u] [-f FILE]
```

hwclock -w 将系统时间设置到时钟芯片里面，此时 rtc 就可以使用了。

注意：如果没有这步，下次启动时，系统时间是不会更新的。

```
/ # hwclock -w
```

断电 过一段时间

上电

```
/ # date
```

已经设置好当前时间。

第四章 OK1052-C 平台 Linux 开发篇

请不要跳过这段话：

开发环境是开发人员在开发过程当中，所需的软硬件平台。开发环境并不是一个固定的样式，在这里，我们详细讲解一个嵌入式 Linux 开发环境搭建的方法。您已经对嵌入式开发非常了解的话，可以按照自己的需求来搭建环境。如果和本手册环境不一样而产生报错，您可以从国内一些大 Linux 论坛和网站搜索相关的信息来解决。本册介绍的环境经过飞凌的测试，如果对嵌入式开发不是非常熟悉的朋友，希望您按照飞凌提供的方法来搭建环境。

4.1 版本说明

- 操作系统：Ubuntu14.04 64 位版
- 交叉工具链：arm-v7-linux-uclibceabi-threads-20170818
- 内核版本：linux4.5

4.2 编译环境说明

Linux PC：在 PC 上直接安装 ubuntu14.04 / ubuntu12.04 作为**编译**的主机，这样做的好处是编译速度快、稳定。

Vmware8.0+ubuntu14.04/Ubuntu 12.04：在 WIN7/XP 下安装虚拟机，并在虚拟机下进行**编译**。这种方式的好处是 1 台电脑就可以搭建开发环境，编译、烧写都在同一台电脑完成，缺点是编译速度太慢，各种网络服务不稳定。

另外，本公司的网盘中提供了带编译器的虚拟机，可作为参考。

4.3 编译环境搭建

依赖包安装步骤如下：

步骤 1：打开 ubuntu 终端输入如下命令，所有命令字符是英文输入字符。

```
$ sudo apt-get install fakeroot
```

```
$ sudo apt-get install mtd-utils
```

```
$ sudo apt-get install lib32z1 #如果交叉编译使用 64bit 系统安装 32bit 兼容包
```

安装交叉编译器步骤如下：

步骤 1：将文件 arm-v7-linux-uclibceabi-threads-20170818.tar.bz2 linux-cortexm-2.5.0.tar.gz 拷贝到 Ubuntu \$HOME/目录下

📁 OK1052-C 用户资料\工具\ arm-v7-linux-uclibceabi-threads-20170818.tar.bz2

步骤 2：在 Ubuntu 中新建一个终端，输入下面的命令安装交叉编译器：

注意：以下命令一条一条输入，命令中所有字符都是**英文状态**下的字符输入，

```
$ cd $HOME
```

```
$ mkdir workspace
```

```
$ tar xvf $HOME/linux-cortexm-2.5.0.tar.gz -C workspace
```

```
$ tar xvf arm-v7-linux-uclibceabi-threads-20170818.tar.bz2 -C workspace/linux-cortexm-2.5.0/tools/
```

```
$ cd $HOME/workspace/linux-cortexm-2.5.0
```

```
$ ls
```

输出如下：

```
A2F_ACTIVATE.sh tools
```

```
$ ls tools/
```

输出如下：

arm-v7-linux-uclibceabi

输入命令：

```
$ tools/arm-v7-linux-uclibceabi/bin/arm-v7-linux-uclibceabi-gcc -v
```

如下输出说明安装成功

gcc version 4.7.4 20130508 (prerelease) (20170818-165657- build on build.emcraft by build)

步骤 3: 在终端里面执行以下命令，验证交叉编译器安装是否成功

配置环境

```
$source ACTIVATE.sh
```

验证

```
$arm-v7-linux-uclibceabi-gcc -v
```

下图为安装成功提示

```
Using built-in specs.
COLLECT_GCC=arm-v7-linux-uclibceabi-gcc
COLLECT_LTO_WRAPPER=/home/disk/home/bluesky/work/workspace/linux-cortexm-2.5.0/tools/arm-
m-v7-linux-uclibceabi/4.7.4/lto-wrapper
Target: arm-v7-linux-uclibceabi
Configured with: /home/build/fdpic/scratch/gcc/configure --prefix=/home/build/fdpic/install
--gmp=/home/build/fdpic/install_host --with-mpfr=/home/build/fdpic/install_host --with-mpc=
root=/home/build/fdpic/install/sysroot --disable-libgomp --enable-libmudflap --enable-lang
le-multilib --disable-libstdcxx-pch --enable-threads=posix --with-arch=armv7-m --with-mode
sable-nls --enable-libstdcxx-time --with-pkgversion='20170818-165657- build on build.emcra
Thread model: posix
gcc version 4.7.4 20130508 (prerelease) (20170818-165657- build on build.emcraft by build)
```

回车，就可以使用该编译器来编译应用代码了。

注意：每次登陆后新打开终端编译应用前都要配置下环境。

```
$source ACTIVATE.sh
```

4.4 编译应用

- 1、进入\$HOME/workspace/linux-cortexm-2.5.0/

建立工程结构文件夹如下：

```
$cd $HOME/workspace/linux-cortexm-2.5.0/
```

```
$mkdir -p projects/rootfs/
```

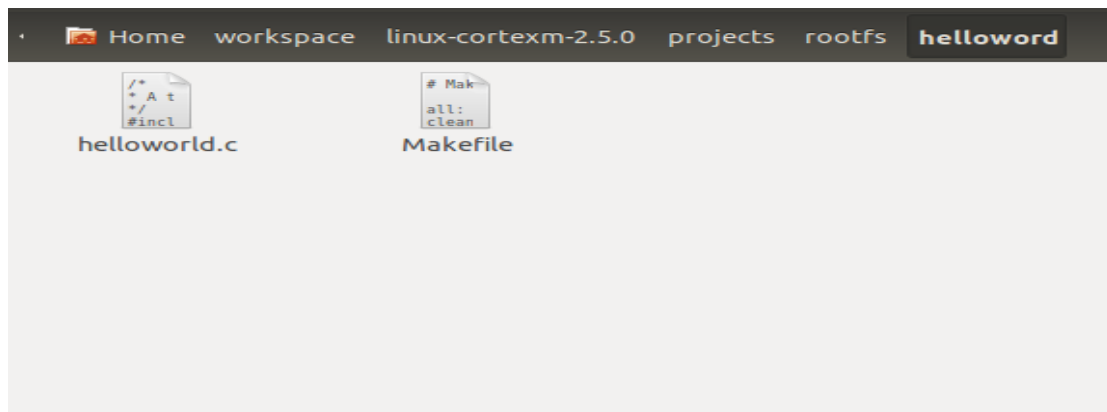
- 2、建立 helloworld 工程

```
$mkdir helloworld
```

新建文件 helloworld.c

新建文件 Makefile

如下图：



helloworld.c 内容如下:

```
helloworld.c ✕
/*
 * A test application that helloworld
 */
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("uclinux helloworld \n");
    return 0;
}
```

Makefile 内容如下:

```
Makefile ✕
# Makefile for the faulting user space programs

all          : apps
clean        : clean_apps

CC = $(CROSS_COMPILE_APPS)gcc
CFLAGS      := -Os -mcpu=cortex-m3 -mthumb -I$(INSTALL_ROOT)/A2F/root/usr/
include
LDFLAGS     := -mcpu=cortex-m3 -mthumb -L$(INSTALL_ROOT)/A2F/root/usr/lib -
static

# Edit the line below to modify a set of user-space programs
# you need to build
APPS        = helloworld
apps        : $(APPS)

%.o:        %.c
             $(CC) -c $(CFLAGS) -o $@ $^

# Clean-up after user-space programs
clean_apps  :
             -rm -f $(APPS) *.gdb *.o
```

Makefile 解释:

CROSS_COMPILE_APPS 交叉编译器前缀

INSTALL_ROOT 依赖库的安装路径

工程文件在 [OK1052-C-Linux 用户资料 \(工业级\) \Linux\测试程序\testapp.zip](#) 中 app-cmd/helloworld

3、编译

配置环境

```
$ cd $HOME/workspace/linux-cortexm-2.5.0/
```

```
$ source ACTIVATE.sh
```

编译

```
$ cd projects/rootfs/helloworld
```

```
$ make CROSS_COMPILE_APPS=arm-v7-linux-uclibceabi- \
INSTALL_ROOT=$HOME/workspace/linux-cortexm-2.5.0
```

4、复制到评估板执行

```
/ # /mnt/sdcard/helloworld
uclinux helloworld
```

4.5 文件系统镜像制作

步骤 1: 将文件 `rootfs-build-tmpdir.tar.bz2` 拷贝到 Ubuntu `$HOME/`目录下

📁 OK1052-C 用户资料\Linux\源码\rootfs-build-tmpdir.tar.bz2 是文件位置

步骤 2: 释放文件,制作 jffs2 文件镜像

```
$ cd $HOME
$ tar xvf rootfs-build-tmpdir.tar.bz2
$ sudo ./make_jffs2_image.sh
```

`rootfs_flash.jffs2` 是生产的文件能烧写到评估板 `norflash` 中。

必须是使用 `ubuntu14.04 64 bit LTS` 制作, 其他版本制作可能遇到制作工具不兼容问题, 需要客户自行解决。

用户自定义应用程序 `helloworld` 安装到文件系统 `rootfs_flash.jffs2` 方法

步骤 1: 复制 `helloworld` 到解压缩后的 `rootfs-build-tmpdir/usr/bin/helloworld`

步骤 2: 执行制作 `rootfs_flash.jffs2` 脚本文件, 生成 `rootfs_flash.jffs2`.

```
$ sudo ./make_jffs2_image.sh
```

附录

F.1 使用 TF 卡启动/烧写镜像

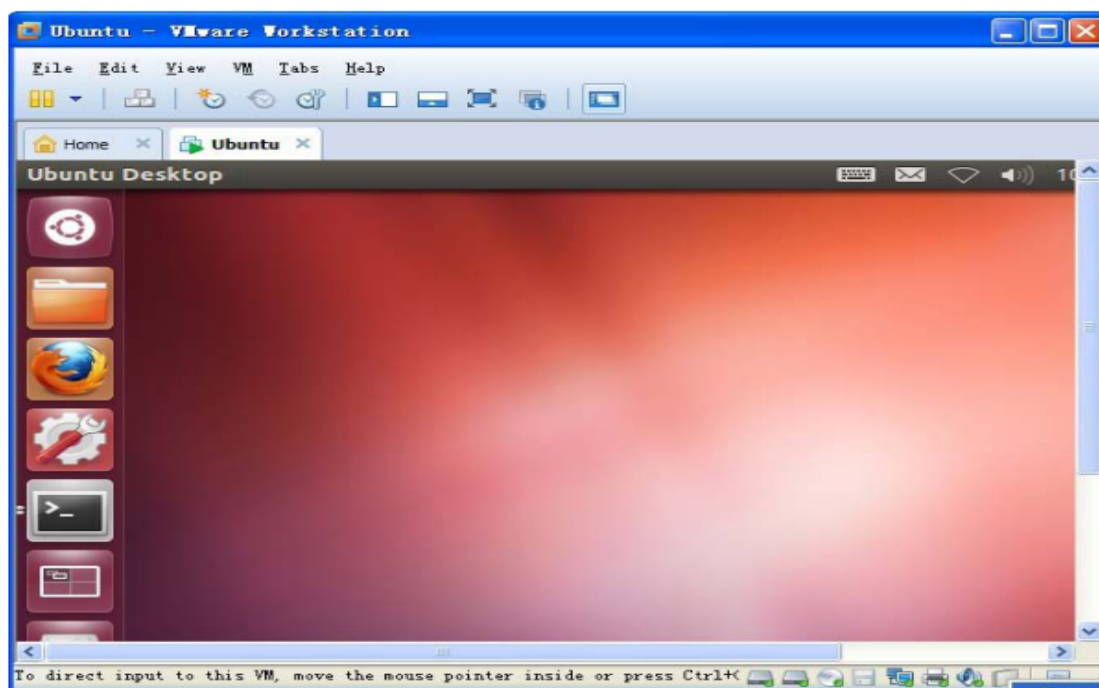
开发板若无系统，第一次需用 TF 卡烧写。

步骤1：制作 TF 卡(Ubuntu 系统下)

📁 路径：OK1052-C 用户资料\Linux\工具\TF\烧写

将此路径下的文件 createSdcard-console.tar.gz 拷贝到到 ubuntu 12.04/Ubuntu 14.04 系统的任一目录，假设为/home/forlinx/workspace。

注意：64bit 系统需要安装 32bit 兼容包。



插入 TF 卡（TF 卡连接到虚拟机中）切换到/home/forlinx/workspace 目录，执行脚本

```
$ cd /home/forlinx/workspace/
```

```
$ tar xvf createSdcard-console.tar.gz
```

```
$ cd createSdcard-console/
```

```
$ sudo ./createSdcard.sh
```

输入 sudo 权限密码

```
This script will create a bootable SD card from custom or pre-built binaries.

The script must be run with root permissions and from the bin directory of
the SDK
```

Example:

```
$ sudo ./create-sdcard.sh
```

```
Formatting can be skipped if the SD card is already formatted and
partitioned properly.
```

```
#####
```

```
Available Drives to write images to:
```

```
# major  minor    size  name
1:    8         16 3872256 sdb
```

```
Enter Device Number: █
```

输入卡编号1

```
# major  minor    size  name
1:    8         16 3872256 sdb
```

```
Enter Device Number: 1
```

```
sdb was selected
```

```
Checking the device is unmounted
```

```
sdb1  sdb2  sdb3
```

```
SD Card is not correctly partitioned
```

```
#####
```

```
Now making 1 partitions
```

```
#####
```

```
1+0 records in
```

```
1+0 records out
```

```
1024 bytes (1.0 kB) copied, 0.0116487 s, 87.9 kB/s
```

```
Disk /dev/sdb doesn't contain a valid partition table
```

```
DISK SIZE - 3965190144 bytes
```

```
Checking that no-one is using this disk right now ...
```

```
OK
```

等待制卡完成。

```
Syncing....
```

```
Un-mount the partitions
```

```
Remove created temp directories
```

```
Operation Finished
```

注：一般情况下，TF 卡只需要制卡处理一次。

步骤2：拷贝镜像文件

📁 路径：OK1052-C 用户资料\Linux\镜像

将镜像文件夹下的 rootfs_flash.jffs2 rootfs_flash.dtb rootfs_flash.ulmage u-boot.flexspi
complete.bmp splash-rt1050-series_24.bmp

复制到 TF 显示的 FAT 分区内

步骤3: 插入 TF 卡, **断电**状态将拨码开关调到 SD/TF 卡启动 (**TF 启动/烧写系统**), 开启电源。

LCD 屏幕显示烧写完成

说明烧写完成。

烧写完成后**断电**,将拨码开关调到 NOR FLASH 启动 (**SPI-NorFlash 启动**) 即可启动系统。

注意: **TF 启动/烧写系统, SPI-NorFlash 启动**, 详见 2.1 章节**设置启动模式**。拨码开关拨动必须**断电**, 否则可能**烧毁 cpu**。